PERSISTENCIA DE DATOS EN FICHEROS

- FICHEROS DE TEXTO PLANO
    + TRABAJAR CON FICHEROS SIN ESTRUCTURA
    + TRABAJAR CON FICHEROS DE ANCHURA DEFINIDA
    + TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV
    + XML
    + HTML
    + JSON
    + YAML
    + LIBRERÍAS CON TODO INCLUIDO (tablib y pandas)
- FICHEROS BINARIOS
    + SERIALIZACIÓN DE OBJETOS PYTHON
    + PERSISTIENDO DICCIONARIOS (shelve)
- COMPRESIÓN Y ARCHIVACIÓN DE FICHEROS
    + COMPRIMIR FICHEROS EN ARCHIVOS ZIP (zipfile)
    + ARCHIVACIÓN Y COMPRESIÓN DE FICHEROS (tarfile)

FICHEROS DE TEXTO PLANO

Se guarda el contenido en texto plano: el contenido no está oculto y se accede simplemente abriendo el fichero.
Tenemos cuatro categorías:

- Ficheros sin estructura básica: no presentan ninguna estructura interna. Son texto guardado en un fichero.
- Ficheros con ancho fijo: presentan un ancho definido para cada campo. Se parecen a tablas donde cada columna tiene definida su anchura en caracteres y se rellenan con espacios cuando es necesario.
- Ficheros tabulados: tienen delimitadores para separar líneas y columnas dentro de las líneas. Por ejemplo, el formato CSV tiene las filas separadas por saltos de línea "\n" y las columnas separadas por comas ",".
- Ficheros en un formato estándar: el propósito es poder presentar la información de forma coordinada con quien la quiere leer.

TRABAJAR CON FICHEROS SIN ESTRUCTURA

Para trabajar con ficheros sin estructura utilizamos la función open():

open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)

Esta función devuelve un objeto de tipo file (descriptor de fichero). Los argumentos:

- file: fichero (dirección absoluta o relativa) | descriptor de fichero

  Si se pasa un descriptor de fichero y closefd es False el objeto descriptor permanece abierto al terminar la función open(). En otro caso, por defecto, se devuelve cerrado.

- mode: modo de acceso al fichero. Puede ser una combinación de:
  + 'r': modo de lectura (por defecto). Si no se combina con otro es lectura de texto plano o rt.
  + 'w': modo de escritura. Escribe un fichero pero trunca el contenido anterior.
  + 'x': modo apertura exclusivamente para nueva creación. Si el fichero ya existe eleva una excepción.
  + 'a': modo de escritura añadiendo al final del fichero.
  + 'b': apertura en modo binario para lectura (rb) y escritura (wb). Devuelve el contenido en objetos tipo bytes.
  + 't': apertura en modo texto para lectura (r o rt) y escritura (w o wt). Devuelve el contenido en objetos de tipo str.
  + '+': apertura en modo actualización para lectura y escritura.
- buffering: opcional. Define una política de buffering:
  + buffering=0: deshabilita el buffering. Solo se usa en archivos binarios.
  + buffering=1: se usa una línea para buffering. Solo se usa en archivos de texto.
  + buffering>1: indica la cantidad de bytes a usar para el buffering e ir leyendo por bloques del fichero.

TRABAJAR CON FICHEROS SIN ESTRUCTURA

open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)
Continuando con los argumentos:

- encoding: codificación a usar para la apertura del fichero. Si no se indica, se usa la del SOP en la variable locale.getpreferredencoding(). Si se abre un fichero en modo binario no se debe especificar la codificación.
- errors: opcional. Permite especificar el tratamiento de errores en las operaciones encode y decode al acceder a un fichero. No se usa en archivos binarios. Los valores posibles son:
    + strinct: eleva una excepción ValueError ante un problema de codificación. La opción por defecto None tiene el mismo efecto.
    + ignore: ignora cualquier problema de codificación y omite el carácter problemático. Se pierde información.
    + replace: reemplaza los caracteres problemáticos con un marcador '?'.
    + surrogateescape: ante un problema de codificación se presentan los bytes incorrectos como caracteres Unicode en un rango reservado para ello (U+DC80 - U+DFF). Si se usa este mismo modo en la función inversa (lectura o escritura), se vuelve a mostrar el caracter original. Es útil cuando no se sabe la codificación del fichero.
    + xmlcharrefreplace: solo soportado cuando se escribe un fichero. Al detectar un error reemplaza el carácter por el carácter XML de referencia.
    + backslashreplace: se escapan los caracteres confictivos utilizando '\'.
    + namereplace: reemplaza los caracteres conflictivos con la representación nominal del carácter en la tabla Unicode.

TRABAJAR CON FICHEROS SIN ESTRUCTURA

open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)

Continuando con los argumentos:

- newline: define como analizar los saltos de línea. Solo se aplica a los ficheros de texto. Caracteres: None, '', '\n', '\r' ' o '\r\n'. Se usan de la siguiente forma:

  Lectura de ficheros:

  + None: habilita el modo de saltos de línea universal. Las líneas que terminen en '\n', '\r' o '\r\n' se transforman en '\n'.
  + '': se devuelven las líneas sin modificar el salto de línea especificado en el fichero.
  + '\n', '\r' o '\r\n': los saltos de línea estarán delimitados y reconocidos por el caracter especificado.

  Escritura de ficheros:

  + None: cualquier caracter '\n' al caracter especificado como separador por defecto del sistema encontrado en os.linesep.
  + '' o '\n': no se hará ninguna transformación en los saltos de línea.
  + '\r' o '\r\n': cualquier carácter '\n' será transformado en el caracter especificado en newline.

- closefd: si el parámetro file es un descriptor y closefd es False, cuando se cierre el fichero se mantendrá abierto el descriptor. Cuando el parámetro file es una dirección de fichero, este parámetro debe permanecer a True (valor por defecto).

TRABAJAR CON FICHEROS SIN ESTRUCTURA

Veamos como se pueden hacer lecturas y escrituras de ficheros y cómo escribiendo un contenido con codificación Latin-1 e intentándolo leer en UTF-8 se pueden manejar los errores de manera diferente:

```
# escribimos en un fichero con codificación 'latin-1' y leemos en 'UTF-8'
# manejando los errores de forma diferente
nombre_fichero = 'mi_fichero.txt'
f = open(nombre_fichero, 'w', encoding='latin-1')
f.write('El árbol es marrón') # 18 caracteres escritos
f.close()
r = open(nombre_fichero, 'r', encoding='latin-1')
print(r.read())

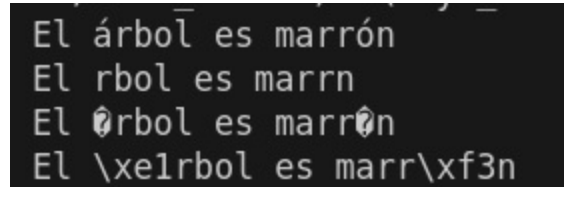
#r = open(nombre_fichero, 'r', encoding='utf-8') #UnicodeDecodeError
r = open(nombre_fichero, 'r', encoding='utf-8', errors='ignore')
print(r.read())

r = open(nombre_fichero, 'r', encoding='utf-8', errors='replace')
print(r.read())

r = open(nombre_fichero, 'r', encoding='utf-8', errors='backslashreplace')
print(r.read())

#r = open(nombre_fichero, 'r', encoding='utf-8', errors='namereplace') #TypeError
#r = open(nombre_fichero, 'r', encoding='utf-8', errors='xmlcharrefreplace') #TypeError
#r = open(nombre_fichero, 'r', encoding='utf-8', errors='surrogateescape')
#UnicodeEncodeError
```

Realiza la actividad
Ficheros_sin_estructura.py



TRABAJAR CON FICHEROS SIN ESTRUCTURA

La función open() devuelve:

- objeto TextIOWrapper para archivos de texto.
- objeto BufferedWriter para archivos binarios.

Este tipo de objetos se usan como clase base para flujos de datos de ficheros, memoria y conexiones de red.

TextIOWrapper y BufferedWriter heredan de BufferedIOBase y IOBase. Por tanto, comparten muchas funciones. Las más utilizadas son:

- IOBase.readable(): devuelve True si el flujo de datos se puede leer. False en caso contrario.
- IOBase.read(size=-1): lee size bytes o caracteres (o los que queden, si quedan menos). Si size es negativo o no se especifica, devuelve toda la cantidad disponible. Si devuelve 0 significa que el fichero se ha leido completamente. Si el fichero no permite lectura devuelve una excepción.
- IOBase.readline(size=-1): lee la línea siguiente. Para ficheros binarios el delimitador será siempre b'\n' (byte string) y para los de texto se especifica en newline (de open())
- IOBase.readlines(hint=-1): lee una serie de líneas del flujo de datos (como máximo lo que indica hint). El delimitador de líneas en los binarios es b'\n'. Retorna una list[str].
- IOBase.readall(): solo para archivos binarios. Devuelve todos los bytes de un flujo de datos hasta el final del fichero.
- IOBase.seekable(): devuelve True si el flujo soporta acceso aleatorio. False en otro caso.

TRABAJAR CON FICHEROS SIN ESTRUCTURA

Continuando con las funciones ... :

- IOBase.seek(offset, whence=SEEK_SET): cambia la posición actual en el flujo de datos para colocarla en una posición nueva. La posición nueva = whence + offset:
    + SEEK_SET o 0: (por defecto) el offset comienza a partir del inicio del flujo de datos. offset >= 0.
    + SEEK_CUR o 1: el offset se calcula a partir de la posición actual. 0 <= offset <= 0.
    + SEEK_END o 2: el offset se calcula a partir del final del flujo de datos. offset <= 0.
- IOBase.tell(): devuelve la posición actual en el flujo de datos.
- IOBase.writable(): devuelve True si el flujo de datos permite escritura. False en otro caso.
- IOBase.write(cadena o bytes): escribe la cadena o bytes pasados como argumentos y devuelve el número de caracteres o bytes escritos (depende si ha sido abierto en modo texto o binario).
- IOBase.writelines(lines): realiza la escritura de una lista de líneas en el flujo de escritura. Es usual que cada línea tenga un caracter de finalización de línea al final.
- IOBase.flush(): fuerza la escritura de los flujos de escritura.
- IOBase.truncate(size=None): permite cambiar el tamaño de un flujo de datos a size bytes. Si no se especifica el tamaño, se reducirá a 0 bytes. Si el nuevo tamaño es mayor que el actual, se añadirán ceros para rellenar el nuevo espacio.

TRABAJAR CON FICHEROS SIN ESTRUCTURA

Continuando con las funciones:

- IOBase.readinto(b): lee el número de bytes especificado en b. Solo para flujos binarios.
- IOBase.close(): realiza un flush y cierra el flujo de datos. Cualquier intento de lectura o escritura en un flujo cerrado eleva una excepción de tipo ValueError.
- IOBase.closed: propiedad que permite comprobar si un flujo está cerrado (True) o no (False).

```
# ejemplo de como copiar líneas impares de un fichero a otro
def copia_impares(fichero_origen, fichero_destino):
    with open(fichero_origen, 'r', newline='') as fr:
        with open(fichero_destino, 'w', newline='') as fw:
            idx = 0
            for linea in fr.readlines():
                if not idx % 2:
                    fw.write(linea)
                idx += 1
```

Una buena forma de usar open() es con el encapsulador de contextos with ya que los flujos de datos que abre open() tienen definido como se deben cerrar los ficheros al usar este tipo de contextos.
NOS EVITAMOS cerrar explícitamente cada flujo de datos abierto.

```
def examinar_contenido(nombre_fichero):
    with open(nombre_fichero, 'r') as f:
        return f.read()

fichero_origen = 'english.txt'
fichero_destino = 'english_impares.txt'
copia_impares(fichero_origen, fichero_destino)
print(examinar_contenido(fichero_destino))
```

english.txt

```
1. On the other hand, we denou
2. who are so beguiled and de
3. so blinded by desire, that
4.  and equal blame belongs t
5. which is the same as sayin
6. These cases are perfectly
7. In a free hour, when our p
8. our being able to do what
9. But in certain circumstanc
10. it will frequently occur
11. The wise man therefore alw
12. he rejects pleasures to s
```

Realiza la actividad copia_impares.py

Realiza la actividad copia_impares2.py

english_impares.txt

```
1. On the other hand, we denounce with righteous indignation and dislike men
3. so blinded by desire, that they cannot foresee the pain and trouble that are bound to ensue;
5. which is the same as saying through shrinking from toil and pain.
7. In a free hour, when our power of choice is untrammelled and when nothing prevents
9. But in certain circumstances and owing to the claims of duty or the obligations of business
11. The wise man therefore always holds in these matters to this principle of selection:
```

TRABAJAR CON FICHEROS SIN ESTRUCTURA

```
# ejemplo de como copiar trozos de un determinado tamaño de un
# fichero a otro saltado en el contenido con un desplazamiento definido
def copiar_trozos(fichero_origen, fichero_destino, saltos=100, tamano=10):
    with open(fichero_origen, 'r', newline='') as fr:
        with open(fichero_destino, 'w', newline='') as fw:
            line = fr.read(tamano)
            while line:
                fw.write(line)
                #Avanzando en el fichero
                new_seek = fr.tell() + saltos
                fr.seek(new_seek)
                #leyendo 10 caracteres
                line = fr.read(tamano)


def examinar_contenido(nombre_fichero):
    with open(nombre_fichero, 'r') as f:
        return f.read()
```

```
fichero_origen = 'english.txt'
fichero_destino = 'english_saltos.txt'
copiar_trozos(fichero_origen, fichero_destino)
print(examinar_contenido(fichero_destino))
```

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$
KINGSTON/UNED_PYTHON/8. EyS_Ficheros/copiar_trozos.py"
1. On the alized by d trouble s of will, simple an prevents
9. But ifrequentlyays holds leasures,
```

Cambiamos la posición del cursor de lectura

Realiza la actividad copiar_trozos.py

english.txt

```
1. On the other hand, we denounce with righteous indignation and dislike men
2. who are so beguiled and demoralized by the charms of pleasure of the moment,
3. so blinded by desire, that they cannot foresee the pain and trouble that are bound to ensue;
4.  and equal blame belongs to those who fail in their duty through weakness of will,
5. which is the same as saying through shrinking from toil and pain.
6. These cases are perfectly simple and easy to distinguish.
7. In a free hour, when our power of choice is untrammelled and when nothing prevents
8. our being able to do what we like best, every pleasure is to be welcomed and every pain avoided.
9. But in certain circumstances and owing to the claims of duty or the obligations of business
10. it will frequently occur that pleasures have to be repudiated and annoyances accepted.
11. The wise man therefore always holds in these matters to this principle of selection:
12. he rejects pleasures to secure other greater pleasures, or else he endures pains to avoid worse pains.
```

TRABAJAR CON FICHEROS DE ANCHURA DEFINIDA

Son ficheros de texto plano con un formato particular. Sus valores internos se distribuyen por cada línea del fichero con una longitud fija a modo de columnas de una tabla.

Al saber de antemano la longitud de caracteres de cada fila y las posiciones en las que se puede encontrar cada valor, se pueden escanear los ficheros tanto horizontal como verticalmente y extraer rápidamente un valor específico.

Para el manejo de este tipo de ficheros se usan parseadores:

- manualmente: debemos programar manualmente los anchos y el tipo de datos de cada columna para el parser.
- mediante la librería struct: trabaja a bajo nivel y es muy rápido. Es muy tedioso de trabajar. No lo vemos.
- mediante la librería pandas: es la opción más recomendable. Incorpora una función para crear DataFrames desde ficheros de ancho fijo especificando solo el tamaño de cada campo. Los tipos se infieren automáticamente.

TRABAJAR CON FICHEROS DE ANCHURA DEFINIDA

```
from  dataclasses import dataclass


@dataclass
class Info: #clase para mapear los valores del fichero
    amigos: int
    nombre: str
    salario: int
    def __post_init__(self):
        self.nombre = self.nombre.strip()


def ancho_fijo_manual(fichero_origen, anchos):
    info = []
    headers = None
    with open(fichero_origen, 'r', newline='') as fr:
        for line in fr.readlines():
            if headers is None:
                headers = line
                continue #Omite la primera línea de cabeceras


            inicio, fin = 0, anchos[0]
            amigos = int(line[inicio:fin])
            inicio, fin = anchos[0], sum(anchos[:2])
            nombre = line[inicio:fin]
            inicio, fin = sum(anchos[:2]), sum(anchos[:3])
            salario = int(line[inicio:fin])
            info.append(Info(amigos, nombre, salario))
    return info
```

```
fichero_origen = 'ejemplo_ancho_fijo.txt'
anchos = [6, # Núm. amigos
          17, # Nombre
          7] # Salario
print(ancho_fijo_manual(fichero_origen, anchos))
```

ejemplo_ancho_fijo.txt

```
  AmigosNombre           Salario
     34  Manuel Ortega     90214
   3452    Juan Martín     17232
    256     Ana Pastor     45324
   7345  Pedro Sánchez     90643
   2163 María Fernández    23419
```

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$ /bin/python3 "/media/natxo/
KINGSTON/UNED_PYTHON/8. EyS_Ficheros/mapeo_anchura_definida_manual.py"
[Info(amigos=34, nombre='Manuel Ortega', salario=90214), Info(amigos=3452, nombre='Juan Martín',
 salario=17232), Info(amigos=256, nombre='Ana Pastor', salario=45324), Info(amigos=7345, nombre=
'Pedro Sánchez', salario=90643), Info(amigos=2163, nombre='María Fernández', salario=23419)]
```

Realiza la actividad
mapeo_anchura_definida_manual.py

TRABAJAR CON FICHEROS DE ANCHURA DEFINIDA

```
# una mejora del anterior es pasar ancho de columna y el tipo al parser
from  dataclasses import dataclass

@dataclass
class Info: #clase para mapear los valores del fichero
    amigos: int
    nombre: str
    salario: int
    def __post_init__(self):
        self.nombre = self.nombre.strip()

def ancho_fijo_generico(fichero_origen, anchos_con_tipo):
    info = []
    headers = None
    with open(fichero_origen, 'r', newline='') as fr:
        for line in fr.readlines():
            if headers is None:
                headers = line
                continue # Omite la primera linea de cabeceras
            acc = 0
            variables = []
            for ancho, tipo in anchos_con_tipo:
                inicio, fin = acc, acc + ancho
                valor = tipo(line[inicio:fin])
                variables.append(valor)
                acc += ancho
            info.append(Info(*variables))
    return info
```

conversión

expandimos la lista ya que el constructor de Info necesita lo de dentro de la lista

```
fichero_origen2 = 'ejemplo_ancho_fijo2.txt'
anchos_con_tipo = [
                (6, int), # Núm. amigos
                (16, str), # Nombre
                (7, int) # Salario
                ]
print(ancho_fijo_generico(fichero_origen2,
anchos_con_tipo))
```

ejemplo_ancho_fijo2.txt

	Amigos	Nombre	Salario
2	34	Manuel Ortega	90214
3	3452	Juan Martín	17232
4	256	Ana Pastor	45324
5	7345	Pedro Sánchez	90643
6	2163	María Fernández	23419

Esta versión peca de ser algo lenta ya que la conversión se hace en Python.

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$ /bin/python3 "/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros/mapeo_anchura_definida_manual2.py"
[Info(amigos=34, nombre='Manuel Ortega', salario=90214), Info(amigos=3452, nombre='Juan Martín', salario=17232), Info(amigos=256, nombre='Ana Pastor', salario=45324), Info(amigos=7345, nombre='Pedro Sánchez', salario=90643), Info(amigos=2163, nombre='María Fernández', salario=23419)]
```

Realiza la actividad mapeo_anchura_definida_manual2.py

TRABAJAR CON FICHEROS DE ANCHURA DEFINIDA

```python
import pandas as pd

def ancho_fijo_pandas(fichero_origen3, anchos):
    return pd.read_fwf(fichero_origen3, widths=anchos)

anchos = [6, 16, 7]  # Amigos Nombre Salario

fichero_origen3 = "ejemplo_ancho_fijo3.txt"
info_df = ancho_fijo_pandas(fichero_origen3, anchos)
print(info_df)
print(info_df.mean(axis=0, numeric_only=True))
```

ejemplo_ancho_fijo3.txt

```
AmigosNombre          Salario
    34  Manuel Ortega   90234
  3452     Juan Martin   17232
   256      Ana Pastor   45324
  7345  Pedro Sanchez   90643
  2163Maria Fernandez   23419
```

Realiza la actividad
mapeo_anchura_definida_pandas.py

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$ /bin/python3 "/
media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros/mapeo_anchura_definida_pandas.py"
   Amigos           Nombre  Salario
0      34    Manuel Ortega    90234
1    3452       Juan Martin    17232
2     256        Ana Pastor    45324
3    7345    Pedro Sanchez    90643
4    2163  Maria Fernandez    23419
Amigos      2650.0
Salario    53370.4
dtype: float64
```

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

Se usan delimitadores como separadores entre valores. Los caracteres más usados son:

- La coma ',': formato CSV (comma-separated values). Extensión .csv
- El tabulador: formato TSV (tab-separated values). Extensión .tsv

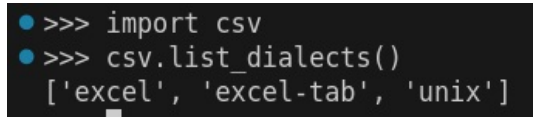
Para trabajar con este tipo de ficheros Python cuenta con la librería csv que permite especificar los delimitadores y los caracteres que rodean a cada valor.

MÉTODOS Y FUNCIONES DEL MÓDULO CSV

- csv.reader(csvfile, dialect='excel', **fmtparams): devuelve un objeto iterador para iterar sobre las líneas de csvfile.
    + csvfile: iterador que devuelva una cadena de caracteres por iteración | objeto tipo fichero abierto con newline = ''.
    + dialect: dialecto para escanear el fichero. La lista se puede obtener de csv.list_dialects(). Se pueden registrar nuevos con csv.register_dialect.
    + fmtparams: parámetros extra para sobreescribir propiedades del dialecto seleccionado.

NOTA: la lectura del contenido se hace iterando con bucles for o con llamadas a next(obj).

- csvreader.line_num: devuelve el número de líneas leidas desde el iterador.
- csvreader.fieldnames: si no se especifica la lista de fields en la creación del objeto, se inicializará la lista de campos cuando se acceda por primera vez al fichero o cuando se lea la primera línea del mismo.



TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

MÉTODOS Y FUNCIONES DEL MÓDULO CSV

- csv.writer(csvfile, dialect='excel', **fmtparams): permite escribir información delimitada y en forma de cadenas de caracteres en el objeto csvfile.

    + csvfile: objeto que implemente el método write() | objeto tipo fichero abierto en modo escritura con newline=''.

    + dialect: dialecto a usar para el escaneo del fichero ...

    + fmtparams: parámetros extra para sobreescribir propiedades del dialecto seleccionado.

NOTA: cuando se escriben valores None se guardan como cadenas de caracteres vacías.

- csvwriter.writerow(row): guarda en el fichero los valores del iterable row teniendo en cuenta el dialecto elegido en el constructor de csvwriter. Devuelve el resultado de la llamada al método write() del objeto usado para guardar información.

- csvwriter.writerows(rows): rows es un iterador de rows y guarda cada row en el fichero, en una nueva línea (según el dialecto elegido en el constructor)

- csv.list_dialects(): ['excel', 'excel-tab', 'unix']

- class csv.DictReader(f, fieldnames=None, restkey=None, restval=None, dialect='excel', *args, **kwargs): devuelve un objeto tipo reader que permite leer la información de un fichero en forma de mapas. Desde la versión 3.8 son objetos dict.

    + f: flujo de datos | descriptor de fichero con el que se pretende trabajar.

    + fieldnames; secuencia de campos que determinan las claves. Si se omite este parámetro se generan a partir de la primera línea del fichero. El orden se conserva durante la lectura de todo el fichero.

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

MÉTODOS Y FUNCIONES DEL MÓDULO CSV

+ restkey: si alguna fila tiene más valores que los de fieldnames, se guardan en la lista especificada en restkey.
+ restval: si alguna fila tiene menos valores que los especificados en fieldnames, se añade el valor de restval a los que falten. Por defecto es None.
+ dialect: especifica el dialecto que se usará para la lectura de los valores del fichero.
+ args y kwargs: especifican valores extra para el dialecto elegido.

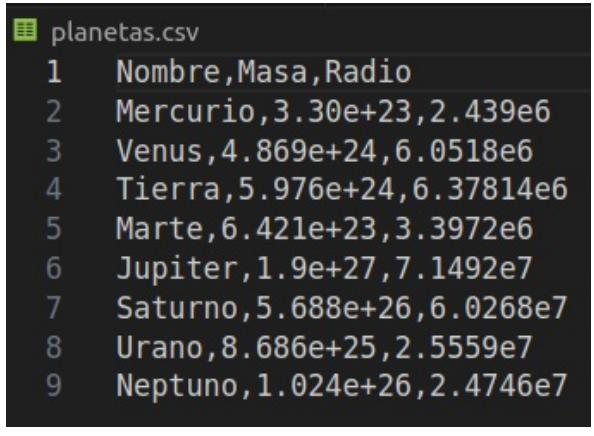
- csv.DictWriter(f, fieldnames, restval='', extrasaction='raise', dialect='excel', *args, **kwargs): devuelve un objeto tipo writer que permite escribir información en el fichero en forma de mapas.
  + f: flujo de datos | descriptor de fichero para trabajar.
  + fieldnames: secuencia de campos que determinan las claves que deben escribirse en el fichero
  + restval: si a algún diccionario le falta una clave de las especificadas en fieldnames, se añadirá este valor en la posición adecuada.
  + extrasaction: especifica el comportamiento a seguir cuando falta alguna clave en un diccionario. Por defecto, 'raise' eleva una excepción ValueError. Con 'ignore', el error es ignorado y se escribe el valor de restval.
  + dialect: especifica el dialecto que se usará para la escritura de los valores en el fichero.
  + args y kwargs: especifican valores extra para el dialecto elegido.
- DictWriter.writeheader(): método de los objetos tipo DictWriter. Guarda una fila con las cabeceras del fichero usando fieldnames.

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

MÉTODOS Y FUNCIONES DEL MÓDULO CSV

- class csv.Sniffer: clase que permite deducir el formato del fichero.
    + sniff(sample, delimiters=None): analiza un fragmento del archivo y devuelve el dialecto (presunto).
    + has_header(sample): analiza un fragmento del archivo. Devuelve True si en la primera fila aparecen las cabeceras del archivo. False en otro caso.

En los siguientes ejemplos analizaremos el fichero planetas.csv:



TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

Ejemplo: lectura fichero csv

```
import csv

def lectura_csv(nombre_fichero):
    '''abre un fichero en formato csv y lo
    muestra por pantalla
    '''
    with open(nombre_fichero, 'r') as fr:
        reader = csv.reader(fr) # obtenemos iterador sobre las líneas del fichero csv
        for fila in reader:
            print(fila) # list

lectura_csv('planetas.csv')
```

Realiza la actividad ejemplo_csv.py

```
['Nombre', 'Masa', 'Radio']
['Mercurio', '3.30e+23', '2.439e6']
['Venus', '4.869e+24', '6.0518e6']
['Tierra', '5.976e+24', '6.37814e6']
['Marte', '6.421e+23', '3.3972e6']
['Jupiter', '1.9e+27', '7.1492e7']
['Saturno', '5.688e+26', '6.0268e7']
['Urano', '8.686e+25', '2.5559e7']
['Neptuno', '1.024e+26', '2.4746e7']
```

Los valores obtenidos son de tipo cadena. Por defecto, no se infieren tipos de datos.

Mediante una clase dataclass o namedtuple se puede añadir tipado y alguna funcionalidad.

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

Ejemplo: lectura de fichero csv con función generadora

```
import csv
from dataclasses import dataclass


@dataclass
class Planeta:
    nombre: str
    masa: float
    radio: float

    def __post_init__(self):
        self.radio = float(self.radio)
        self.masa = float(self.masa)

    @property
    def gravedad(self):
        g_constante = 6.673e-11
        return g_constante * self.masa / (self.radio ** 2)
```

Conversiones ya que el reader solo devuelve str

Atributo de solo lectura

Realiza la actividad ejemplo_csv_tipado.py

Para ficheros grandes es conveniente usar un generador para la lectura ya que ocupan muy poca memoria RAM.

```
def lectura_de_planetas(nombre_fichero):
    '''función generadora: cada vez que se llame devuelve objeto Planeta
    producido a partir de una fila del fichero'''
    with open(nombre_fichero, 'r') as fr:
        reader = csv.reader(fr) # iterador de filas (list[str]) del csv
        tiene_cabecera = csv.Sniffer().has_header(fr.read(1024))
        fr.seek(0) # coloca la posición de lectura al inicio de nuevo
        if tiene_cabecera:
            cabecera = next(reader) # ignora la cabecera
        for fila in reader:
            yield Planeta(*fila) # descomponemos la list


if __name__ == '__main__':
    for p in lectura_de_planetas('planetas.csv'): # generador de Planetas
        print(f'La gravedad en {p.nombre} es {p.gravedad}')
```

```
La gravedad en Mercurio es 3.701787325376329
La gravedad en Venus es 8.871391908774457
La gravedad en Tierra es 9.802652743337129
La gravedad en Marte es 3.7126290961053403
La gravedad en Jupiter es 24.80617666947324
La gravedad en Saturno es 10.44978014597121
La gravedad en Urano es 8.8726476241638
La gravedad en Neptuno es 11.158634802412358
```

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

Ejemplo: ampliación del anterior. Se escribe un nuevo fichero csv de planetas con la columna extra "diametro". Se hace uso de DictWriter.

```
import csv
from dataclasses import dataclass

@dataclass
class Planeta: ...

def lectura_de_planetas(nombre_fichero): ...

def escritura_de_planetas(nombre_fichero, fichero_salida):
    with open(fichero_salida, 'w') as fw:
        columnas = ['nombre', 'masa', 'radio', 'diámetro']
        writer = csv.DictWriter(fw, fieldnames=columnas, extrasaction='ignore')
        writer.writeheader()
        for planeta in lectura_de_planetas(nombre_fichero):
            # __dict__ in Python represents a dictionary or any mapping object
            #  that is used to store the attributes of the object. mappingproxy objects
            valores = planeta.__dict__
            valores['diámetro'] = planeta.radio * 2
            writer.writerow(valores)

if __name__ == '__main__':
    escritura_de_planetas('planetas.csv', 'nuevos_planetas.csv')
```

Realiza la actividad ejemplo_csv_tipado_escritura.py

nuevos_planetas.csv

```
1  nombre,masa,radio,diámetro
2  Mercurio,3.3e+23,2439000.0,4878000.0
3  Venus,4.869e+24,6051800.0,12103600.0
4  Tierra,5.976e+24,6378140.0,12756280.0
5  Marte,6.421e+23,3397200.0,6794400.0
6  Jupiter,1.9e+27,71492000.0,142984000.0
7  Saturno,5.688e+26,60268000.0,120536000.0
8  Urano,8.686e+25,25559000.0,51118000.0
9  Neptuno,1.024e+26,24746000.0,49492000.0
10
```

TRABAJAR CON FICHEROS EN FORMATO CSV Y TSV

Ejemplo: lectura escritura ficheros csv con la librería pandas. Ventaja: se pueden inferir los tipos de las columnas y puede especificarse el tratamiento de tipos de datos complejos como las fechas.

```
import pandas as pd

def lectura_escritura_pandas(fichero_entrada, fichero_salida):
    df = pd.read_csv(fichero_entrada) # lectura fichero

    # se aplican las modificaciones en el dataframe
    print(df.head()) # muestra las primeras líneas del fichero

    # este cálculo lo hace para todo el dataframe. Tipo álgebra vectorial
    df['Diámetro'] = df['Radio'] * 2 # calcula el diámetro.

    df.to_csv(fichero_salida, index=False) # escribe el fichero

lectura_escritura_pandas('planetas.csv', 'planetas_pandas.csv')
```

Realiza la actividad ejemplo_csv_pandas.py

planetas_pandas.csv

```
Nombre,Masa,Radio,Diámetro
Mercurio,3.3e+23,2439000.0,4878000.0
Venus,4.869e+24,6051800.0,12103600.0
Tierra,5.976e+24,6378140.0,12756280.0
Marte,6.421e+23,3397200.0,6794400.0
Jupiter,1.9e+27,71492000.0,142984000.0
Saturno,5.688e+26,60268000.0,120536000.0
Urano,8.686e+25,25559000.0,51118000.0
Neptuno,1.024e+26,24746000.0,49492000.0
```

Para cambios en pequeños documentos el uso de pandas es excesivo ya que el paquete pandas es bastante grande.
Si se pretende hacer muchas modificaciones de datos es recomendable usarlo. Pandas dispone de muchas herramientas para manejo de datos: agregaciones por columnas, operaciones matemáticas complejas, ...

https://aprendeconalf.es/docencia/python/manual/pandas/

XML

Los archivos csv interesan cuando se guardan datos que tienen la misma estructura y no hay jerarquías.

Para datos con jerarquías se usan otros formatos como el XML (eXtensive Markup Language).

El formato XML:

- Permite definir jerarquías de datos.
- Es independiente de la plataforma en que se use.
- Permite intercambio de información multiplataforma (BDA, apps) mediante el fichero XML.

El formato XML tiene dos componentes principales: las etiquetas y los atributos.

Ejemplo: planetas.xml

Jerarquía (etiquetas): sistemasolar -> planeta (atributo nombre) -> masa, radio (atributo unidad)

```
<sistemasolar>
    <planeta nombre="Mercurio">
        <masa>3.303e23</masa>
        <radio unidad="km">2.4397e6</radio>
    </planeta>
    <planeta nombre="Venus">
        <masa>4.869e+24</masa>
        <radio unidad="km">6.051e6</radio>
    </planeta>
    <planeta nombre="Tierra">
        <masa>5.976e+24</masa>
        <radio unidad="km">6.37814e6</radio>
    </planeta>
    <planeta nombre="Marte">
        <masa>6.421e+23</masa>
        <radio unidad="km">3.397e6</radio>
    </planeta>
    <planeta nombre="Jupiter">
        <masa>1.9e+27</masa>
        <radio unidad="km">7.1492e7</radio>
    </planeta>
    <planeta nombre="Saturno">
        <masa>5.688e+26</masa>
        <radio unidad="km">6.0268e7</radio>
    </planeta>
    <planeta nombre="Urano">
        <masa>8.686e+25</masa>
        <radio unidad="km">2.5559e7</radio>
    </planeta>
    <planeta nombre="Neptuno">
        <masa>1.024e+26</masa>
        <radio unidad="km">2.4746e7</radio>
    </planeta>
</sistemasolar>
```

XML

La librería estándar para trabajar con ficheros XML es xml.etree.ElementTree y se importa como ET para abreviar. Las funciones más utilizadas:

- ET.parse(source, parser=None): devuelve un objeto tipo ElementTree extraido del parámetro source.
    + source: fichero | objeto file.
    + parser: parseador a usar. Por defecto se usa XMLParser.
- ET.SubElement(parent, tag, attrib={}, **extra): crea subelementos que se añaden a parent
    + tag: nombre del elemento
    + attrib: dict con los atributos que deberá tener.
    + extra: atributos extra añadidos como clave=valor
- ET.fromstring(text, parser=None): parsea la cadena de caracteres text en un objeto tipo Element. Por defecto, se usa el parser XMLParser.
- class ET.Element(tag, attrib={}, **extra): representa un elemento XML con el nombre de la etiqueta (tag), los atributos en attrib como dict o en extra como clave=valor. Los métodos y atributos disponibles:
    + attrib: dict con todos los atributos del elemento.
    + tag: nombre del elemento.
    + get(key, default=None): devuelve el valor del atributo con nombre key. Si no se encuentra devuelve el valor del parámetro default.
    + find(match, namespaces=None): encuentra el primer subelemento cuyo nombre sea igual que match. Si match es una ruta devuelve el primer elemento de esa ruta. Se pueden usar namespaces para filtrar la búsqueda a un solo espacio de nombres específico.

XML

- class ET.Element ...
    + findall(match, namespaces=None): similar a find pero devuelve una lista de subelementos.
    + findtext(match, default=None, namespace=None): busca en el elemento y en los subelementos hasta que encuentre uno que sea igual que match. Entonces devuelve el texto asociado al elemento encontrado. El parámetro match puede ser el identificador del elemento o una ruta donde encontrarlo.
    + iterfind(match, namespaces=None): similar a findall pero devuelve un iterador en lugar de una lista.
    + itertext(): crea un iterador que va devolviendo todos los textos internos de ese elemento y de los subelementos.
- class ET.ElementTree(element=None, file=None)
    + find, findall, findtext, iterfind: similar a Element pero aplicado a ElementTree.
    + getroot(): devuelve el elemento raíz del árbol construido.
    + write(file, enconding="us-ascii", xml_declaration=None, default_namespace=None, method="xml", *, short_empty_elements=True): permite la escritura del arbol asociado en un fichero XML pasado en file. Los demás parámetros son descriptivos. En method se puede especificar "xml", "text" o "html".

XML

Ejemplo: lectura e importación de datos de planetas.xml a objetos tipo Planeta

```
import xml.etree.ElementTree as ET
from dataclasses import dataclass

@dataclass
class Planeta: ...

def extraer_planetas_de_xml(fichero_planetas):
    planetas = []
    xml = ET.parse(fichero_planetas)
    planetas_xml = xml.findall('planeta') # retorna list(Element)
    for planeta_xml in planetas_xml:
        planetas.append(extraer_planeta_de_xml(planeta_xml))
    return planetas

def extraer_planeta_de_xml(planeta_xml):
    nombre = planeta_xml.attrib['nombre']
    masa = planeta_xml.find('masa').text
    radio = planeta_xml.find('radio').text
    return Planeta(nombre, masa, radio)

if __name__ == '__main__':
    planetas = extraer_planetas_de_xml('planetas.xml')
    for p in planetas:
        print(f'Nombre: {p.nombre} gravedad: {p.gravedad}')
```

Realiza la actividad
ejemplo_importar_archivos_xml.py

Obtenemos objeto ElementTree: <sistema solar>

Obtenemos lista de subelementos

Obtenemos un objeto Planeta de cada subelemento <planeta>

Lista de objetos Planeta

Element.attrib es un diccionario de atributos del elemento

Buscamos el primer subelemento <masa>

text es el texto entre <masa> y </masa>

Generamos objeto con dataclass

```
Nombre: Mercurio gravedad: 3.7030267229659395
Nombre: Venus gravedad: 8.873737829343
Nombre: Tierra gravedad: 9.802652743337129
Nombre: Marte gravedad: 3.713066274602545
Nombre: Jupiter gravedad: 24.80617666947324
Nombre: Saturno gravedad: 10.44978014597121
Nombre: Urano gravedad: 8.8726476241638
Nombre: Neptuno gravedad: 11.158634802412358
```

XML

Ejemplo: exportar el contenido a un archivo XML desde los objetos Planeta ya creados. Se añade la gravedad de cada planeta en el archivo XML.

```
import xml.etree.ElementTree as ET
import ejemplo_importar_archivos_XML

def escribir_planetas(planetas, fichero_salida):
    raiz = ET.Element('sistemasolar-completo') # raíz del árbol final

    for planeta in planetas:
        # crea Element planeta con attrib nombre
        planeta_xml = ET.Element('planeta', attrib=dict(nombre=planeta.nombre))
        # crea Element masa
        masa_xml = ET.Element('masa') #agrega cada elemento de cada planeta
        masa_xml.text = str(planeta.masa)
        # crea Element radio
        radio_xml = ET.Element('radio', atrib=dict(unidad='km'))
        radio_xml.text = str(planeta.radio)
        # crea Element gravedad
        gravedad_xml = ET.Element('gravedad')
        gravedad_xml.text = str(planeta.gravedad)
        # se añaden subelementos al planeta
        planeta_xml.extend([masa_xml, radio_xml, gravedad_xml])
        # se engancha el planeta a la raíz
        raiz.append(planeta_xml)

    arbol = ET.ElementTree(raiz) #crea un objeto arbol para poder guardar
    arbol.write(fichero_salida) #guarda el árbol completo en un fichero

if __name__ == '__main__':
    planetas = ejemplo_importar_archivos_XML.extraer_planetas_de_xml('planetas.xml')
    fichero_xml_salida = 'planetas_completos.xml'
    escribir_planetas(planetas, fichero_xml_salida)
```

Hay que definir cada elemento del árbol e ir construyendo nodo a nodo. En https://docs.python.org/3/library/xml.etree.elementtree.html se puede encontrar info de todas las funciones.

planetas_completos.xml

```
<sistemasolar-completo>
    <planeta nombre="Mercurio">
        <masa>3.303e+23</masa>
        <radio atrib="{'unidad': 'km'}">2439700.0</radio>
        <gravedad>3.7030267229659395</gravedad>
    </planeta>
        <planeta nombre="Venus">
            <masa>4.869e+24</masa>
            <radio atrib="{'unidad': 'km'}">6051000.0</radio>
            <gravedad>8.873737829343</gravedad>
    </planeta>
```

```
    <planeta nombre="Neptuno">
        <masa>1.024e+26</masa>
        <radio atrib="{'unidad': 'km'}">24746000.0</radio>
        <gravedad>11.158634802412358</gravedad>
    </planeta>
</sistemasolar-completo>
```

Realiza la actividad ejemplo_exportar_archivos_XML.py

HTML

HTML (Hyper Text Markup Language) es un estándar para describir páginas web creado por W3C.

Es semejante a XML ya que tiene etiquetas y jerarquías pero en el caso de HTML cada etiqueta tiene un propósito definido.

Cada etiqueta está definida como <nombre_etiqueta> y la mayoría están obligadas a cerrarse </nombre_etiqueta>.

Las etiquetas pueden contener texto y atributos clave-valor. Los valores son de tipo str (rodeados por comillas).

El HTML declara cómo debe representarse la información por medio de texto. Esto permite elaborar un documento que represente una página web en un simple archivo de texto.

Ejemplo: planetas.html

```
<html>
    <head>
        <title>Planetas del sistema solar</title>
        <style>
            body {text-align: center;}
            h1 {color: grey;}
            h2 {color: navy;}
            .masa {font-weight: bold}
            footer {background: gray; color: white;
                margin: 20px;padding: 20px;}
        </style>
    </head>
    <body>
        <h1>Sistema Solar</h1>
        <div class="planeta">
            <h2>Mercurio</h2>
            <div class="masa">3.303e+23</div>
            <div class="radio">2439700</div>
        </div>
       ...
```

```
...
 <div class="planeta">
   <h2>Neptuno</h2>
   <div class="masa">1.024e+26</div>
   <div class="radio">24746000</div>
 </div>
</body>
<footer>
  <div>
    Python a fondo
  </div>
</footer>
</html>
```

HTML

Para parsear archivos HTML se pueden usar diferentes librerías de Python.

La librería más básica es html.parser que permite crear un parseador de HTML personalizado iterando por los elementos del documento.

Ejemplo:

```
from html.parser import HTMLParser

class HTMLParserPropio(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print(f'Comienzo de etiqueta: {tag}')

    def handle_endtag(self, tag):
        print(f'Fin de etiqueta: {tag}')


    def handle_data(self, data):
        info = data.strip()
        if info:
            print(f'Contenido: {info}')

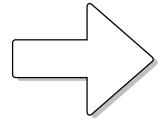
parser = HTMLParserPropio()
parser.feed('<html><head><title>Ejemplo simple</title></head>'
            '<body><h1>ejemplo especial</h1></body>')
```

HTMLParserPropio hereda de HTMLParser.

El contenido se analiza con la función feed.

Para más info:

https://docs.python.org/es/3.10/library/html.parser.html



```
Comienzo de etiqueta: html
Comienzo de etiqueta: head
Comienzo de etiqueta: title
Contenido: Ejemplo simple
Fin de etiqueta: title
Fin de etiqueta: head
Comienzo de etiqueta: body
Comienzo de etiqueta: h1
Contenido: ejemplo especial
Fin de etiqueta: h1
Fin de etiqueta: body
```

Realiza la actividad ejemplo_html_parser.py

HTML

La librería lxml (https://lxml.de/) es la más rápida ya que está escrita en C:

- Tiene soporte para analizar documentos XML y HTML.
- Permite utilizar la mayoría de funciones de la librería estándar ElementTree.
- Soporta búsquedas XPath.
- Permite transformar archivos XML a XSLT (eXtensive Stylessheet Language Transformations). Es decir, permite transformar XML a otros formatos.

Ejemplo: parsear fichero planetas.html

```
from lxml import etree

with open('planetas.html') as fs:
    cadena_html = fs.read()

html = etree.HTML(cadena_html) # parseamos a HTML
result = etree.tostring(html, pretty_print=True, method="html")
print(result.decode())
```

Sin el decode saca bytes: b'...'

etree.HTML(): parsea una cadena a HTML y retorna el nodo raíz

etree.tostring(): serializa un elemento (nodo raíz) a un encoded string que representa el arbol HTML. La codificación por defecto es ASCII.

Realiza la actividad ejemplo_lxml1.py

```
<html>
  <head>
    <title>Planetas del sistema solar</title>
    <style>
      body {text-align: center;}
      h1 {color: grey;}
      h2 {color: navy;}
      .masa {font-weight: bold}
      footer {background: gray; color: white;
        margin: 20px;padding: 20px;}
    </style>
  </head>
  <body>
    <h1>Sistema Solar</h1>
    <div class="planeta">
      <h2>Mercurio</h2>
      <div class="masa">3.303e+23</div>
      <div class="radio">2439700</div>
    </div>
    ...
    <div class="planeta">
      <h2>Neptuno</h2>
      <div class="masa">1.024e+26</div>
      <div class="radio">24746000</div>
    </div>
  </body>
  <footer>
    <div>
      Python a fondo
    </div>
  </footer>
</html>
```

HTML

lxml permite crear un parseador propio especificando para que etiqueta se debe usar.

Ejemplo: parser personalizado para eventos de comienzo y fin de etiqueta y de parseo de contenido.

```
from lxml import etree
class HTMLParserPropio(object):
    def start(self, tag, attrs):
        print(f'Comienzo de etiqueta: {tag}')

    def end(self, tag):
        print(f'Fin de etiqueta: {tag}')

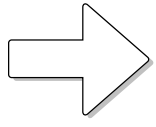
    def data(self, data):
        info = data.strip()
        if info:
            print(f'Contenido: {info}')

    def comment(self, text):
        print(f'Comentario: {text}')

    def close(self):
        print('Cerrando ...')

with open('planetas.html', 'r') as fs:
    cadena_html = fs.read()

parser = etree.XMLPullParser(target=HTMLParserPropio())
parser.feed(cadena_html)
parser.close()
```



```
Comienzo de etiqueta: html
Comienzo de etiqueta: head
Comienzo de etiqueta: title
Contenido: Planetas del sistema solar
Fin de etiqueta: title
Comienzo de etiqueta: style
Contenido: body {text-align: center;}
        h1 {color: grey;}
        h2 {color: navy;}
        .masa {font-weight: bold}
        footer {background: gray; color: white;
          margin: 20px;padding: 20px;}
Fin de etiqueta: style
Fin de etiqueta: head
Comienzo de etiqueta: body
Comienzo de etiqueta: h1
Contenido: Sistema Solar
Fin de etiqueta: h1
Comienzo de etiqueta: div
Comienzo de etiqueta: h2
Contenido: Mercurio
Fin de etiqueta: h2
Comienzo de etiqueta: div
Contenido: 3.303e+23
Fin de etiqueta: div
Comienzo de etiqueta: div
Contenido: 2439700
Fin de etiqueta: div
Fin de etiqueta: div
...
Fin de etiqueta: body
Comienzo de etiqueta: footer
Comienzo de etiqueta: div
Contenido: Python a fondo
Fin de etiqueta: div
Fin de etiqueta: footer
Fin de etiqueta: html
Cerrando ...
```

Realiza la actividad ejemplo_lxml_parser.py

HTML

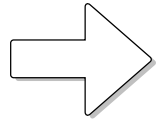
Ejemplo: lxml con búsqueda de elementos con XPath (XML Path Language)

```
from lxml import etree

with open('planetas.html', 'r') as fs:
    cadena_html = fs.read()
    parse = etree.fromstring(cadena_html)
    nombre_planetas = parse.xpath('//div[@class="planeta"]/h2')

    for nombre in nombre_planetas:
        print(nombre.text)
```

parsea la cadena de caracteres cadena_html en un objeto tipo Element



Mercurio
Venus
Tierra
Marte
Jupiter
Saturno
Urano
Neptuno

Realiza la actividad
ejemplo_lxml_xpath.py

Para más información sobre XPath:

https://www.mclibre.org/consultar/xml/lecciones/xml-xpath.html

HTML

La librería de más alto nivel para HTML es BeautifulSoup que usa html.parser, lxml o html5lib como parsers y permite:

- Interactuar con el documento y acceder a los elementos mediante .getattr (".")
- Conversión a Unicode o incluso la detección de la codificación de los documentos de forma nativa.
- Navegación de un nodo hacia nodos padres, hijos y hermanos de forma intuitiva.
- Búsqueda en el árbol con diferentes métodos: expresiones regulares, listas de etiquetas, palabras clave, funciones propias, búsquedas usando clases CSS, búsquedas en el texto de las etiquetas, ...
- Buscar en el documento mediante selectores CSS.
- Crear nodos con funciones.
- Trabajar con ficheros XML y HTML con facilidad.
- Parsear solo partes de un documento.

Para saber más:

https://pypi.org/project/beautifulsoup4/

Ejemplo: parseo del documento planetas.html con BeautifulSoup, cálculo de la gravedad de cada planeta, se añade un nuevo nodo con dicha información en cada planeta y se guarda todo en un fichero HTML nuevo.

HTML

```python
from bs4 import BeautifulSoup, Tag

def calc_gravedad(masa, radio):
    g_constante = 6.67e-11
    return g_constante * float(masa) / (float(radio) ** 2)

with open('planetas.html', 'r') as fs:
    cadena_html = fs.read()

html = BeautifulSoup(cadena_html, 'html.parser')
print(html.title.text)
print(html.body.find_all('h2'))
planetas = html.find_all('div', class_='planeta')
for planeta in planetas:
    masa = planeta.select_one('.masa').text
    radios = [x for x in planeta.children if isinstance(x, Tag) and ['radio'] == x.attrs.get('class')]
    radio = radios[0].text
    gravedad = calc_gravedad(masa, radio)
    gravedad_tag = html.new_tag(name='div', attrs={'class': 'gravedad'})
    gravedad_tag.string = str(gravedad)
    planeta.append(gravedad_tag)

nombre_salida = 'planetas_con_gravedad.html'
with open(nombre_salida, 'w') as fw:
    fw.write(html.prettify())
```

Realiza la actividad ejemplo_beautifulsoup.py

Planetas del sistema solar

[<h2>Mercurio</h2>,
<h2>Venus</h2>,
<h2>Tierra</h2>,
<h2>Marte</h2>,
<h2>Jupiter</h2>,
<h2>Saturno</h2>,
<h2>Urano</h2>,
<h2>Neptuno</h2>]

BeautifulSoup se usa mucho para búsquedas (mediante selectores CSS) o para manipular el documento HTML.

BeautifulSoup no soporta XPath como lxml pero se pueden usar en conjunción.

Una buena combinación es usar BeautifulSoup con lxml como parseador.

Si se busca velocidad lxml es más rápida ya que está escrita en C.

planetas_con_gravedad.html > html

```html
<html>
 <head>
  <title>
   Planetas del sistema solar
  </title>
  <style>
   body {text-align: center;}
            h1 {color: grey;}
            h2 {color: navy;}
            .masa {font-weight: bold}
            footer {background: gray; color: white;
                margin: 20px;padding: 20px;}
  </style>
 </head>
 <body>
  <h1>
   Sistema Solar
  </h1>
  <div class="planeta">
   <h2>
    Mercurio
   </h2>
   <div class="masa">
    3.303e+23
   </div>
   <div class="radio">
```

JSON

El formato JSON (JavaScript Object Notation) se usa mucho en la industria de las aplicaciones web. Su ventaja es que permite crear parsers de contenido de forma rápida. Al igual que XML, permite intercambiar datos entre aplicaciones pero JSON necesita menos caracteres para enviar la misma información. Por esto mismo, se utiliza en aplicaciones web que usan AJAX.

Un documento JSON solo puede contener los siguientes elementos (OJO que son tipos de JavaScript):

- Números: positivos y negativos con "." para la coma. También se permite la notación científica.
- Cadenas de caracteres: con longitud variable y siempre entre "" (comillas dobles).
- Booleanos: true y false.
- Vacío: null que equivale a None en Python.
- Listas: en JSON son arrays de longitud variable con "," para separar las variables y se escriben entre corchetes [] .
- Objetos: {} con pares clave-valor separados por ":".

Ejemplo: planetas.json

```
[
    {
        "nombre": "Mercurio",
        "masa": 3.303e+23,
        "radio": 2439700
    },
    {
        "nombre": "Venus",
        "masa": 4.869e+24,
        "radio": 6051000
    },
    {
        "nombre": "Tierra",
        "masa": 5.976e+24,
        "radio": 6378140
    },
    {
        "nombre": "Marte",
        "masa": 6.421e+23,
        "radio": 3397000
    },
    {
        "nombre": "Jupiter",
        "masa": 1.9e+27,
        "radio": 71492000
    },
    {
        "nombre": "Saturno",
        "masa": 5.688e+26,
        "radio": 60268000
    },
    {
        "nombre": "Urano",
        "masa": 8.686e+25,
        "radio": 25559000
    },
    {
        "nombre": "Neptuno",
        "masa": 1.024e+26,
        "radio": 24746000
    }
]
```

JSON

Se puede reducir aún más el contenido: eliminación de espacios, tabuladores y saltos de línea. Siendo más agresivos, se usa una técnica de mapeo externo que convierte las claves de los objetos en un solo carácter. Las claves "nombre", "masa" y "radio" quedarían "n", "m" y "r". Esto exige que las aplicaciones conozcan el mapeo.

La librería estándar que da soporte a JSON es json. Tiene los siguientes métodos:

- json.load(fp, *, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw): deserializa el contenido de un fichero fp a objetos Python.
    + fp: fichero de texto o binario que contiene el documento JSON.
    + cls: permite usar un decodificador propio (hereda de json.JSONDecoder) y no el que está por defecto.
    + object_hook: permite usar una función propia para convertir los literales del fichero en objetos.
    + object_pairs_hook: similar al anterior pero los literales se pasan a la función como lista de pares.
    + parse_float: permite definir una función para parsear los objetos de tipo float. Por defecto, se usa float(num_str).
    + parse_int: permite definir una función que se usará para parsear los objetos de tipo int. Por defecto, se usa int(num_str).
    + parse_constant: permite definir una función que se usará para definir constantes como '-Infinity', 'Infinity' o 'NaN'.
- json.loads(s, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw): similar a json.load() pero su primer parámetro s es un str.

JSON

- json.dump(obj, fp, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw): serializa un objeto Python obj a un documento JSON escribiendo en el fichero fp.
    + obj: objeto Python a serializar.
    + fp: fichero sonde se guarda el documento JSON.
    + skipkeys: si es True, las claves del diccionario que no sean de un tipo básico son omitidas y no se eleva excepción TypeError. Tipos básicos: str, int, float, bool y None.
    + ensure_ascii: por defecto True y asegura que todos los caracteres no ASCII sean escapados. Con False, se guardan los caracteres tal y como están en Python (puede provocar errores de conversión en aplicaciones que lean su contenido).
    + check_circular: por defecto True y se comprueba si hay objetos circulares (hacen referencia a sí mismos). Como False pueden producirse errores OverflowError o bucles infinitos por recursión.
    + allow_nan: como True (por defecto), convierte los valores Python nan, inf e -inf a sus homólogos JavaScript Nan, Infinity e -Infinity. Como False, eleva excepción ValueError al encontrarse con esos valores.
    + cls: para definir serializadores propios que hereden de la clase json.JSONEncoder.
    + indent: especifica el caracter para indentar cada nueva línea en JSON. Un número representa el número de espacios para indentar. Si se pone 0 o '' solo se hace uso de saltos de línea. Con None se obtiene la versión más compacta (por defecto): sin indentación ni saltos de línea.

JSON

- json.dump(): ...

    + separators: especifica los separadores como una tupla (separador_elemento, separador_clave) y por defecto son (',', ':').

    + default: permite escribir una función que devuelva la versión JSON de cualquier valor que no pueda ser convertido automáticamente.

    + sort_keys: a True, se devuelven los diccionarios ordenados según las claves que contienen. Por defecto es False.

NOTA: el módulo json siempre produce cadenas de caracteres tipo str, por lo que fp debe soportar la escritura de las mismas.

- json.dumps(obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw): retorna una cadena de caracteres en vez de guardar en fp como hace json.dump().

- class json.JSONDecoder(*, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, strict=True, object_pairs_hook=None): clase que especifica el decodificador para hacer las deserializaciones por defecto. Convierte las variables en formato JSON a Python. Esta clase se usa creando una clase que hereda de JSONDecoder y permite gestionar las conversiones para clases propias de Python.

- class json.JSONEncoder(*, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, sort_keys=False, indent=None, separators=None, default=None): clase que especifica el codificador para hacer las serializaciones por defecto. Se utiliza para convertir variables Python en formato JSON. Esta clase se usa creando una clase que hereda de JSONEncoder y permite gestionar las conversiones de clases de Python a objetos JSON.

JSON

Conversiones que hace la librería json:

Formato JSON <----------> Objeto Python	
null	None
true	True
false	False
number (int)	int
number (float)	float
array	list, tuple
object	dict
string	str

Hay que implementar conversores de JSON -> Python y de Python -> JSON cuando se trabaja con objetos propios u objetos del núcleo de Python que no están en esa lista como time, decimal o fractions.

Ejemplo: leer planetas.json, convertir el contenido a objetos tipo Planeta en Python y volver a guardar en otro archivo el contenido y la gravedad de cada Planeta. Como los objetos de tipo Planeta no están soportados por defecto se crean funciones propias para pasar de JSON a Python y de Python a JSON.

JSON

```
import json
from ejemplo_importar_archivos_XML import Planeta

def como_planetas(dct):
    '''Retorna objeto Planeta a partir de un diccionario
    que se le pasa como parámetro
    '''
    return Planeta(dct['nombre'], dct['masa'], dct['radio'])

class PlanetaEncoder(json.JSONEncoder):
    '''Si el objeto es de tipo Planeta lo pasa a dict
    para que json.JSONEncoder pueda codificarlo.
    En otro caso, json.JSONEncoder lo codifica directamente'''
    def default(self, obj):
        if isinstance(obj, Planeta):
            return dict(nombre=obj.nombre, masa=obj.masa, radio=int(obj.radio), gravedad=obj.gravedad)
        return json.JSONEncoder.default(self, obj)

with open('planetas.json', 'r') as fr:
    # deserializa planetas.json a objetos Python
    planetas = json.load(fr, object_hook=como_planetas)
    print(planetas[0])

with open('planetas_con_gravedad.json', 'w') as fw:
    # serializa planetas (objetos Python) a planetas_con_gravedad.json
    json.dump(planetas, fw, cls=PlanetaEncoder, indent=4)
```

La nueva propiedad gravedad se define en PlanetaEncoder de manera que no es necesario añadirla al código.

Planeta(nombre='Mercurio', masa=3.303e+23, radio=2439700.0)

Realiza la actividad ejemplo_json.py



YAML

YAML (YAML Ain't Markup Language) es un formato de documento orientado a dos aspectos:

- Que todos los datos puedan ser representados como literales.
- Legibilidad del documento final.

Se usan guiones:

- para separar elementos de listas (al mismo nivel).
- para elementos anidados (a distintos niveles).

Se usa la indentación de bloques para separar elementos (como Python).

Ejemplo: planetas.yaml

```
- !!python/object:__main__.Planeta
  masa: 3.303e+23
  nombre: Mercurio
  radio: 2439700.0
- !!python/object:__main__.Planeta
  masa: 4.869e+24
  nombre: Venus
  radio: 6051000.0
- !!python/object:__main__.Planeta
  masa: 5.976e+24
  nombre: Tiera
  radio: 6378140.0
- !!python/object:__main__.Planeta
  masa: 6.421e+23
  nombre: Marte
  radio: 2297000.0
```

Lista de 4 planetas. '- ' (espacio detrás)

Cada planeta tiene su masa, nombre y radio.

Objeto propio de Python definido dentro de __main__

Los atributos de cada objeto están indentados por dos espacios. Los atributos se representan como pares clave-valor. Entre clave y valor siempre ': ' (espacio detrás)

Los valores de cada atributo son literales (no necesitan comillas)

Para saber más de YAML: https://es.wikipedia.org/wiki/YAML

YAML

El núcleo de Python no tiene soporte para este tipo de formato. Pero se pueden usar librerías de terceros como PyYaml (https://pypi.org/project/PyYAML/).

Ejemplo: definimos una clase PlanetaYaml que herede de Planeta y de yaml.YAMLObject. De esta forma, se puede especificar la etiqueta que debe utilizar el objeto para su representación en YAML y sus instancias se pueden guardar en fichero en formato YAML.

```python
import yaml
from ejemplo_importar_archivos_XML import Planeta

class PlanetaYaml(Planeta, yaml.YAMLObject):
    yaml_tag = '!Planeta'

    def __init__(self, nombre, masa, radio):
        super(PlanetaYaml, self).__init__(nombre, masa, radio)
        self.__dict__['gravedad'] = self.gravedad

if __name__ == '__main__':
    planetas = []
    planetas.append(PlanetaYaml(nombre='Mercurio', masa=3.303e+23, radio=6051000.0))
    planetas.append(PlanetaYaml(nombre='Venus', masa=4.869e+24, radio=6051000.0))
    planetas.append(PlanetaYaml(nombre='Tierra', masa=5.976e+24, radio=6378140))

    with open('planetas_con_gravedad.yaml', 'w') as fw:
        yaml.dump(planetas, fw)
```

Lo hace prácticamente todo

planetas_con_gravedad.yaml

```yaml
- !Planeta
  gravedad: 0.6019707547816785
  masa: 3.303e+23
  nombre: Mercurio
  radio: 6051000.0
- !Planeta
  gravedad: 8.873737829343
  masa: 4.869e+24
  nombre: Venus
  radio: 6051000.0
- !Planeta
  gravedad: 9.802652743337129
  masa: 5.976e+24
  nombre: Tierra
  radio: 6378140.0
```

Como la librería PyYaml solo guarda atributos de las instancias que se encuentran en __dict__ y gravedad es una propiedad, es necesario añadir la gravedad a __dict__ por ejemplo en __init__ de PlanetaYaml.

La librería PyYaml puede guardar y cargar objetos complejos (time, datetime) sin necesidad de crear un serializador propio.

Realiza la actividad ejemplo_YAML.py

LIBRERÍAS CON TODO INCLUIDO - tablib y pandas

Hay librerías de terceros que permiten trabajar con ficheros en diferentes formatos. Las interfaces para acceder, modificar y guardar esos ficheros son de muy alto nivel (simples). Destacan:

- tablib: https://tablib.readthedocs.io/en/stable/
- pandas: https://pandas.pydata.org/docs/user_guide/index.html#user-guide

Ambas soportan formatos: Excel, JSON, YAML, dataframes de pandas, HTML, Jira, TSV o CSV, ...
pandas, además, soporta formatos: HDF5, docx, mp3, mp4, SQL, ...

Estas librerías convierten los formatos en dataframes (y viceversa). Se pueden relizar operaciones sobre los datos de forma agregada fácilmente.
'tablib' es una librería liviana orientada a la interacción con ficheros de diferentes formatos.
'pandas', librería pesada, hace esa interación de forma lateral (funcionalidad extra) ya que está orientada a la manipulación científica de grandes volúmenes de datos.

Ejemplo: tablib trabajando con planetas.csv

```
import tablib

with open('planetas.csv', 'r') as fr:
    data = tablib.Dataset().load(fr)
    print(data)
```

```
Nombre  |Masa     |Radio
--------|---------|---------
Mercurio|3.30e+23 |2.439e6
Venus   |4.869e+24|6.0518e6
Tierra  |5.976e+24|6.37814e6
Marte   |6.421e+23|3.3972e6
Jupiter |1.9e+27  |7.1492e7
Saturno |5.688e+26|6.0268e7
Urano   |8.686e+25|2.5559e7
Neptuno |1.024e+26|2.4746e7
```

Realiza la actividad ejemplo_tablib.py

LIBRERÍAS CON TODO INCLUIDO - tablib y pandas

Ejemplo: tablib trabajando con planetas.csv

print(data[0])

print(data.export('json'))

print(data.export('yaml'))

print(data.export('df'))

```
('Mercurio', '3.30e+23', '2.439e6')
[{"Nombre": "Mercurio", "Masa": "3.30e+23", "Radio": "2.439e6"}, {"Nombre": "Venus", "M
asa": "4.869e+24", "Radio": "6.0518e6"}, {"Nombre": "Tierra", "Masa": "5.976e+24", "Rad
io": "6.37814e6"}, {"Nombre": "Marte", "Masa": "6.421e+23", "Radio": "3.3972e6"}, {"Nom
bre": "Jupiter", "Masa": "1.9e+27", "Radio": "7.1492e7"}, {"Nombre": "Saturno", "Masa":
 "5.688e+26", "Radio": "6.0268e7"}, {"Nombre": "Urano", "Masa": "8.686e+25", "Radio": "
2.5559e7"}, {"Nombre": "Neptuno", "Masa": "1.024e+26", "Radio": "2.4746e7"}]
- {Masa: '3.30e+23', Nombre: Mercurio, Radio: 2.439e6}
- {Masa: '4.869e+24', Nombre: Venus, Radio: 6.0518e6}
- {Masa: '5.976e+24', Nombre: Tierra, Radio: 6.37814e6}
- {Masa: '6.421e+23', Nombre: Marte, Radio: 3.3972e6}
- {Masa: '1.9e+27', Nombre: Jupiter, Radio: 7.1492e7}
- {Masa: '5.688e+26', Nombre: Saturno, Radio: 6.0268e7}
- {Masa: '8.686e+25', Nombre: Urano, Radio: 2.5559e7}
- {Masa: '1.024e+26', Nombre: Neptuno, Radio: 2.4746e7}

     Nombre       Masa      Radio
0  Mercurio   3.30e+23    2.439e6
1     Venus  4.869e+24   6.0518e6
2    Tierra  5.976e+24  6.37814e6
3     Marte  6.421e+23   3.3972e6
4   Jupiter    1.9e+27   7.1492e7
5   Saturno  5.688e+26   6.0268e7
6     Urano  8.686e+25   2.5559e7
7   Neptuno  1.024e+26   2.4746e7
```

Realiza la actividad ejemplo_tablib.py

LIBRERÍAS CON TODO INCLUIDO - tablib y pandas

Ejemplo: pandas trabajando con planetas.csv

```
import pandas as pd

ds = pd.read_csv('planetas.csv')

print(ds.head())
print(ds.to_json())
print(ds.to_html())
```

```
     Nombre          Masa       Radio
0  Mercurio  3.300000e+23   2439000.0
1     Venus  4.869000e+24   6051800.0
2    Tierra  5.976000e+24   6378140.0
3     Marte  6.421000e+23   3397200.0
4   Jupiter  1.900000e+27  71492000.0
{"Nombre":{"0":"Mercurio","1":"Venus","2":"Tierra","3":"Marte","4":"Jupiter","5":"Satur
no","6":"Urano","7":"Neptuno"},"Masa":{"0":3.3e+23,"1":4.869e+24,"2":5.976e+24,"3":6.42
1e+23,"4":1.9e+27,"5":5.688e+26,"6":8.686e+25,"7":1.024e+26},"Radio":{"0":2439000.0,"1"
:6051800.0,"2":6378140.0,"3":3397200.0,"4":71492000.0,"5":60268000.0,"6":25559000.0,"7"
:24746000.0}}
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>Nombre</th>
      <th>Masa</th>
      <th>Radio</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>Mercurio</td>
      <td>3.300000e+23</td>
      <td>2439000.0</td>
    </tr>
    <tr>
```

Realiza la actividad ejemplo_pandas.py

FICHEROS BINARIOS

Casos de uso:

- si se guarda contenido que no es estrictamente texto, con un formato particular, solo conocido por las aplicaciones que usan ese formato.
- si se usa un formato que puede prescindir de los saltos de línea.

La principal diferencia con los ficheros de texto plano es que su contenido no es fácilmente reconocible.
En multitud de ocasiones contienen datos mucho más complejos que el texto plano: sonidos, imágenes, vídeos, ...
Los datos en los ficheros binarios se agrupan en bytes contiguos y se leen como un stream de datos ya que no suelen tener separadores de bloques de contenido como saltos de línea.

Python no tiene una librería específica para leer datos en binario pero si suele tener soporte para cada formato específico, por lo general, en librerías de terceros.

SERIALIZACIÓN DE OBJETOS EN PYTHON - pickle

'pickle' (https://docs.python.org/3/library/pickle.html) pertenece al conjunto de librerías del núcleo de Python y permite guardar la información de objetos Python directamente en ficheros binarios y cargarla de nuevo en un programa.

La ventajas de poder compartir objetos Python como ficheros (con pickle):

- pueden ser transmitidos fácilmente entre aplicaciones.
- persistencia en disco para ser cargados cuando sea necesario.
- al contrario que JSON y XML, los ficheros son binarios y no son legibles por los humanos.
- al contrario que JSON, pickle soporta cualquier tipo de dato.
- pickle soporta la serialización de funciones Python.

Funciones de pickle:

- class pickle.Pickler(file, protocol=None, *, fix_imports=True, buffer_callback=None): permite escribir un stream de datos pickle en el fichero file.
  + file: variable como fichero que soporte el método write (fichero binario | instancia de io.Bytes)
  + protocol: entero que representa en número de protocolo que se usa para la conversión en pickle. Cambia según versión de Python. Por compatibilidad, no hay que usar un protocolo elevado con versiones antiguas de Python.
  + fix_imports: si se pone a True y protocol es menor que 3 se intenta compatibilidad con Python2.
  + buffer_callback: se usa en serialización, se pasa como argumento un buffer_view si el valor es una función. Añadido en Python 3.8 -> protocolo mínimo 5.
  + def dump(obj): método que permite guardar el objeto obj en su versión pickle en el fichero file según se haya indicado en el objecto Pickler.

SERIALIZACIÓN DE OBJETOS EN PYTHON - pickle

Continuación de las funciones de pickle:

- class pickle.Unpickler(file, * , fix_imports=True, encoding="ASCII", errors= "strict", buffers=None): permite la lectura de un objeto pickle desde un archivo binario.

    + file: objeto del que se lee el contenido. Debe implementar las funciones read(), readinto() y readline(): implementando la API io.BufferedIOBase. Por tanto, debe ser un objeto io.BytesIO o cualquier objeto que cumpla estos requisitos.
    + fix_imports: si es True, intenta mapear módulos importados en Python2 en módulos nuevos Python3.
    + encoding: define la codificación del fichero.
    + errors: define como deben manejarse los errores de importación.
    + buffers: si es None, significa que toda la información sobre la serialización está contenida en file. De lo contrario, se usa la función especificada en este parámetro para deserializar el contenido (desde Python 3.8).

    NOTA: el protocolo que se usó para guardar se detecta automáticamente.

    + def load(): método que convierte el objeto pickle a un objeto Python según se indique en el objeto Unpickler.

- pickle.dump(obj, file, protocol=None, *, fix_imports= True, buffer_callback=None): permite la escritura del objeto obj en su representación pickle en el objeto file. Equivale Pickler(file, protocol).dump(obj) y los parámetros son iguales que en Pickler.
- pickle.dumps(obj, protocol=None, *, fix_imports= True, buffer_callback=None): idem pickle.dump() pero devuelve un objeto como bytes (no escribe en fichero).

SERIALIZACIÓN DE OBJETOS EN PYTHON - pickle

Continuación de las funciones de pickle:

- pickle.load(file, *, fix_imports=True, encoding="ASCII", errors= "strict", buffers=None): permite la lectura de un objeto pickle del fichero y lo pasa a objeto Python. Equivale a Unpickler(file).load() y los parámetros son los mismos que en Unpickler.
- pickle.loads(data, *, fix_imports=True, encoding="ASCII", errors= "strict", buffers=None): permite la lectura del objeto pickle data en un objeto Python. data es un objeto bytes.
- object.__getstate__(): permite influir en cómo se debe hacer la conversión al formato pickle desde cualquier objeto.
- object.__setstate__(state): permite influir en cómo se debe hacer la deserialización de un objeto pickle a un objeto Python desde cualquier objeto.

SERIALIZACIÓN DE OBJETOS EN PYTHON - pickle

Ejemplo: guardamos una lista de objetos Planeta en un fichero binario y lo leemos de nuevo pasando los objetos serializados (pickle) a objetos Python.

```
from ejemplo_csv_tipado import lectura_de_planetas
import pickle

# lectura de planetas de un csv
planetas = list(lectura_de_planetas('planetas.csv'))

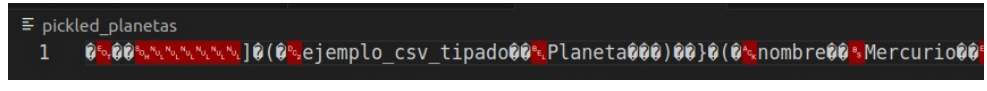
with open('pickled_planetas', 'wb') as fw:
    # preparamos un stream de datos pickle para escribir en fw
    pickler = pickle.Pickler(fw)
    # guarda la versión pickle de planetas en el fichero
    pickler.dump(planetas)

with open('pickled_planetas', 'rb') as fr:
    # prepara stream de lectura desde fr y
    # lee objetos pickle pasándolos a objetos Python
    planetas_deserializados = pickle.Unpickler(fr).load()

print(f'planetas deserializados {planetas_deserializados}')

assert planetas == planetas_deserializados
```

Realiza la actividad ejemplo_pickle.py




```
planetas deserializados [Planeta(nombre='Mercurio', masa=3.3e+23, radio=2439000.0), Planeta(nombre='Venus',
masa=4.869e+24, radio=6051800.0), Planeta(nombre='Tierra', masa=5.976e+24, radio=6378140.0), Planeta(nombre=
'Marte', masa=6.421e+23, radio=3397200.0), Planeta(nombre='Jupiter', masa=1.9e+27, radio=71492000.0), Planet
a(nombre='Saturno', masa=5.688e+26, radio=60268000.0), Planeta(nombre='Urano', masa=8.686e+25, radio=2555900
0.0), Planeta(nombre='Neptuno', masa=1.024e+26, radio=24746000.0)]
```

PERSISTIENDO DICCIONARIOS - shelve

Podemos guardar diccionarios mediante la librería shelve https://docs.python.org/es/3/library/shelve.html).
La única restricción es que las claves de los diccionarios deben ser claves simples. Los valores pueden ser de cualquier tipo aceptado por pickle (librería usada para serializar).

La manera de persistir es casi transparente: simplemente se abre y se cierra el Shelf al comienzo y al final de la ejecución del código.

Funciones disponibles para manejar cada Shelf con la librería shelve:

- shelve.open(filename, flag='c', protocol=None, writeback=False): permite la apertura de la base de datos y devuelve un objeto tipo shelve.Shelf. El parámetro protocol es el número de protocolo de pickle. El parámetro writeback a True especifica si con cada modificación de alguna clave-valor, además de guardar la versión cacheada en memoria, se debe persistir en disco.
- class shelve.Shelf(dict, protocol=None, writeback=False, keyencoding='utf-8'): objeto que la librería shelve usa para manipular la información dentro de la base de datos. Los métodos y manipulación del objeto son similares a los de los diccionarios de Python pero cuando se guarda o modifica la información se hace realmente sobre la base de datos.
- Shelf.sync(): persiste el contenido de la caché en disco. Requiere que se haya abierto el Shelf con writeback a True.
- Shelf.close(): cierra el Shelf y persiste el contenido aún no guardado en la base de datos.

PERSISTIENDO DICCIONARIOS - shelve

Ejemplo: uso básico de shelve.

```
import shelve
from ejemplo_csv_tipado import lectura_de_planetas

nombre_db = 'planetas' # fichero para persistir dicts
d = shelve.open(nombre_db) # abrimos base de datos y obtenemos
objeto Shelf

# genera una lista de objetos Planeta
planetas = list(lectura_de_planetas('planetas.csv'))

# se guarda en el Shelf
d['planetas'] = planetas

# planeta tierra
tierra = [p for p in planetas if p.nombre == 'Tierra'][0]
# lista planetas mayores que la tierra
p_mayores = [p for p in planetas if p.radio > tierra.radio]
# lista planetas menores que la tierra
p_menores = [p for p in planetas if p.radio < tierra.radio]

# se guardan en el Shelf
d['planetas_mayores'] = p_mayores
d['planetas_menores'] = p_menores

d.close() # se cierra y se persisten los cambios

# se abre de nuevo y tiene los datos de antes
d = shelve.open(nombre_db)
print(f'Claves guardadas en shelve: {list(d.keys())}')
for k, v in d.items():
    print(f'{k} -> {v}')
```

Realiza la actividad ejemplo_shelve.py




```
Claves guardadas en shelve: ['planetas_menores', 'planetas_mayores', 'planetas']
planetas_menores -> [Planeta(nombre='Mercurio', masa=3.3e+23, radio=2439000.0),
Planeta(nombre='Venus', masa=4.869e+24, radio=6051800.0), Planeta(nombre='Marte'
, masa=6.421e+23, radio=3397200.0)]
planetas_mayores -> [Planeta(nombre='Jupiter', masa=1.9e+27, radio=71492000.0),
Planeta(nombre='Saturno', masa=5.688e+26, radio=60268000.0), Planeta(nombre='Ura
no', masa=8.686e+25, radio=25559000.0), Planeta(nombre='Neptuno', masa=1.024e+26
, radio=24746000.0)]
planetas -> [Planeta(nombre='Mercurio', masa=3.3e+23, radio=2439000.0), Planeta(
nombre='Venus', masa=4.869e+24, radio=6051800.0), Planeta(nombre='Tierra', masa=
5.976e+24, radio=6378140.0), Planeta(nombre='Marte', masa=6.421e+23, radio=33972
00.0), Planeta(nombre='Jupiter', masa=1.9e+27, radio=71492000.0), Planeta(nombre
='Saturno', masa=5.688e+26, radio=60268000.0), Planeta(nombre='Urano', masa=8.68
6e+25, radio=25559000.0), Planeta(nombre='Neptuno', masa=1.024e+26, radio=247460
00.0)]
```

COMPRESIÓN Y ARCHIVACIÓN DE FICHEROS

La mejor forma de organizar los ficheros en disco es por carpetas. Si se pretende compartir esas carpetas es mejor crear carpetas comprimidas mediante compresores/descompresores.
La carpeta comprimidas son un archivo binario con todos los datos dentro. Por lo general, se reduce el tamaño total de los archivos.
Vamos a ver como se trabaja con archivos en formato ZIP y TAR desde Python.

COMPRIMIR FICHEROS EN ARCHIVOS ZIP - zipfile

Uno de los formatos más utilizados para comprimir archivos en un único archivo contenedor es ZIP (https://es.wikipedia.org/wiki/Formato_de_compresi%C3%B3n_ZIP).

El formato ZIP comprime cada uno de los archivos por separado y luego los agrupa en un solo documento. Esto permite descomprimir cada archivo por separado.

Para trabajar con archivos ZIP en Python se usa la librería zipfile (https://docs.python.org/3/library/zipfile.html). 'zipfile' permite crear, leer y modificar archivos ZIP.

COMPRIMIR FICHEROS EN ARCHIVOS ZIP - zipfile

Funciones de zipfile:

- class zipfile.ZipFile(file, mode='r', compression=ZIP_STORED, allowZip64=True, compresslevel=None, strict_timestamps=True): permite la apertura de un fichero ZIP para leer, guardar o modificar.
    + file: ruta al archivo ZIP | objeto tipo fichero.
    + mode: modo de apertura ('r', 'w', 'a' o 'x').
    + compression: ZIP_STORED (por defecto), ZIP_DEFLATED, ZIP_BZIP2 o ZIP_LZMA. Si no se usa el modo por defecto, las librerías zlib, bz2 y lzma deben estar instaladas respectivamente.
    + allowZip64: permite usar el modo de compresión ZIP64 para ficheros mayores de 4GB. Activado por defecto.
    + compresslevel: de 0 (sin compresión) a 9 (max compresión). Solo tiene efecto con ZIP_DEFLATED o ZIP_BZIP2.
    + strict_timestamps: como False, permite comprimir fichero más antiguos que 1980-01-01 y se usa la fecha límite 1980-01-01. Lo mismo pasa con ficheros posteriores a 2107-12-31 (se usa ésta).
- class zipfile.ZipInfo(filename='NoName', date_time=(1980, 1, 1, 0, 0, 0)): clase usada por zipfile para representar cada elemento dentro de un archivo ZIP. El parámetro filename es el nombre del fichero y date_time es una tupla con la fecha de última modificación del fichero.
- ZipFile.getinfo(name): devuelve un objeto zipfile.ZipInfo que representa un archivo, dentro del ZIP, con nombre name. Si no existe, se eleva una excepción KeyError.
- ZipFile.infolist(): devuelve una lista de objetos zipfile.ZipInfo. Es decir, los archivos dentro del ZIP.
- ZipFile.namelist(): devuelve una lista con los nombres de los archivos dentro del ZIP.

COMPRIMIR FICHEROS EN ARCHIVOS ZIP - zipfile

Funciones de zipfile (continuamos):

- ZipFile.open(name, mode='r', pwd=None, *, force_zip64=False): permite acceder a un fichero con nombre name dentro del ZIP. Devuelve un objeto fichero binario. Parámetro name: nombre fichero | objeto zipfile.ZipInfo. Parámetro mode: 'r' o 'w'. El parámetro pwd especifica la contraseña para abrir (si se necesita). El parámetro force_zip64 se pone a True si esperamos archivos mayores de 4GB para evitar problemas.
- ZipFile.extract(member, path=None, pwd=None): permite extraer un archivo de dentro del ZIP. El archivo lo indicamos con member (nombre | objeto zipfile.ZipInfo). Con path indicamos la carpeta donde se guarda el resultado. Con pwd indicamos la contraseña en caso de ser necesaria.
- ZipFile.extractall(path=None, members=None, pwd=None): permite extraer todo el contenido del ZIP o un subconjunto de archivos (listados con infolist()). El resultado se guarda en la carpeta donde está el fichero ZIP o donde indique path. Con pwd indicamos la contraseña en caso de ser necesaria.
- ZipFile.write(filename, arcname=None, compress_type=None, compresslevel=None): escribe filename en el ZIP, podemos cambiar su nombre con arcname (si es None no cambia pero va sin path). Si ponemos algo de compresión sobreescribe al del constructor.
- ZipFile.setpassword(pwd): permite añadir una contraseña al archivo ZIP.
- ZipFile.close(): cierra el fichero ZIP y guarda el contenido que queda por escribir. Si se usan los modos 'w', 'a' o 'x' y no se hace close() no se guardará el contenido en el fichero ZIP.

Ejemplo: guardamos la versión XML de cada planeta en ficheros separados. Éstos ficheros se guardan en un ZIP. Después leemos solo la información de Neptuno. Luego creamos un ZIP con planetas.xml que tiene la info de todos los planetas. Finalmente, comparamos cual de los 2 ZIP ocupa menos.

COMPRIMIR FICHEROS EN ARCHIVOS ZIP - zipfile

Vamos a generar todos los ficheros xml a partir de planetas.csv:

```
# de aquí se importa lectura de planetas para obtener una lista de objetos Planeta a partir de un csv
from ejemplo_csv_tipado_escritura import lectura_de_planetas
# para construir un arbol de nodos
import xml.etree.ElementTree as ET

def planeta_en_xml(planeta):
    '''
    retorna un elemento planeta_xml con sus elementos hijo
    a partir de un objeto Planeta
    '''
    planeta_xml = ET.Element('planeta')
    for campo in ['nombre', 'masa', 'radio', 'gravedad']:
        campo_xml = ET.Element(campo)
        campo_xml.text = str(getattr(planeta, campo))
        planeta_xml.append(campo_xml)
    return planeta_xml

def escribir_xml(planeta):
    '''se ocupa de escribir el fichero xml de un planeta'''
    planeta_xml = planeta_en_xml(planeta)
    arbol = ET.ElementTree(planeta_xml)
    arbol.write(f'{planeta.nombre}.xml')

def escribir_planetas_xml(planetas):
    planetas_xml = ET.Element('planetas')
    for planeta in planetas:
        planeta_xml = planeta_en_xml(planeta)
        planetas_xml.append(planeta_xml)
    arbol = ET.ElementTree(planetas_xml)
    arbol.write('planetas.xml')
```

```
if __name__ == '__main__':
    # obtenemos lista de objetos Planeta
    planetas = list(lectura_de_planetas('planetas.csv'))
    # escritura del fichero xml de cada planeta
    for planeta in planetas:
        escribir_xml(planeta)
    # escritura del fichero xml de todos los planetas
    escribir_planetas_xml(planetas)
```

Mercurio.xml

```
<planeta>
    <nombre>Mercurio</nombre>
    <masa>3.3e+23</masa>
    <radio>2439000.0</radio>
    <gravedad>3.701787325376329</gravedad>
</planeta>
```

planetas.xml

```
<planetas>
    <planeta>
        <nombre>Mercurio</nombre>
        <masa>3.3e+23</masa>
        <radio>2439000.0</radio>
        <gravedad>3.701787325376329</gravedad>
    </planeta>
    <planeta>
        <nombre>Venus</nombre>
        <masa>4.869e+24</masa>
        <radio>6051800.0</radio>
        <gravedad>8.871391908774457</gravedad>
    </planeta>
    <planeta>
        <nombre>Tierra</nombre>
        <masa>5.976e+24</masa>
        <radio>6378140.0</radio>
        <gravedad>9.802652743337129</gravedad>
    </planeta>
```

Realiza la actividad

generar_un_xml_por_planeta.py

COMPRIMIR FICHEROS EN ARCHIVOS ZIP - zipfile

Ya podemos hacer lo que se pide:

```
import zipfile

if __name__ == '__main__':
    ficheros_planetas = [
        'Mercurio.xml', 'Venus.xml', 'Tierra.xml', 'Marte.xml',
        'Jupiter.xml', 'Saturno.xml', 'Urano.xml', 'Neptuno.xml'
    ]
    nombre_zip = 'zip_planetas_separados.zip'
    fichero_zip = zipfile.ZipFile(nombre_zip, mode='w') # para escribir
    for nombre_fichero_xml in ficheros_planetas:
        fichero_zip.write(nombre_fichero_xml)
    fichero_zip.close() # persistencia

    zip_leido = zipfile.ZipFile(nombre_zip, mode='r') # para leer
    print(zip_leido.namelist())
    print(zip_leido.infolist())

    neptuno = zip_leido.getinfo('Neptuno.xml')
    print(neptuno)
    print(zip_leido.extract('Neptuno.xml', 'Neptuno_extraido'))

    nombre_zip = 'zip_planetas_unico_fichero.zip'
    fichero_zip = zipfile.ZipFile(nombre_zip, mode='w') # para escribir
    fichero_zip.write('planetas.xml')
    fichero_zip.close() # persistencia
```

['Mercurio.xml', 'Venus.xml', 'Tierra.xml', 'Marte.xml', 'Jupiter.xml', 'Saturno.xml', 'Urano.xml', 'Neptuno.xml']

[<ZipInfo filename='Mercurio.xml' filemode='-rw-r--r--' file_size=147>, <ZipInfo filename='Venus.xml' filemode='-rw-r--r--' file_size=125>, <ZipInfo filename='Tierra.xml' filemode='-rw-r--r--' file_size=126>, <ZipInfo filename='Marte.xml' filemode='-rw-r--r--' file_size=126>, <ZipInfo filename='Jupiter.xml' filemode='-rw-r--r--' file_size=126>, <ZipInfo filename='Saturno.xml' filemode='-rw-r--r--' file_size=128>, <ZipInfo filename='Urano.xml' filemode='-rw-r--r--' file_size=124>, <ZipInfo filename='Neptuno.xml' filemode='-rw-r--r--' file_size=129>]

<ZipInfo filename='Neptuno.xml' filemode='-rw-r--r--' file_size=129>

Neptuno_extraido/Neptuno.xml

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$ ll zip*
-rw-r--r-- 1 natxo natxo 1825 jul 25 10:37 zip_planetas_separados.zip
-rw-r--r-- 1 natxo natxo 1522 jul 25 10:37 zip_planetas_unico_fichero.zip
```

Más comprimido

Realiza la actividad ejemplo_zipfile.py

ARCHIVACIÓN Y COMPRESIÓN DE FICHEROS - tarfile

El formato más utilizado en Unix para archivar varios archivos en uno solo es TAR (Tape ARchive). Existen varias opciones:

Formato corto	Formato largo	Librería de compresión
.tgz	.tar.gz	gzip
.tbz, .tbz2, .tb	.tar.bz2	bzip2
.lz	.tar.lz	lzip
.txz	.tar.xz	xz

Python soporta este tipo de archivadores con o sin compresión mediante la librería tarfile (https://docs.python.org/3/library/tarfile.html). En cuanto a los archivos comprimidos soporta gzip, gzip2 y lzma.

Las funciones de tarfile:

- tarfile.open(name=None, mode='r', fileobj=None, bufsize=10240, **kwargs): permite abrir el archivo TAR especificado en name y lo convierte en un objeto tarfile.TarFile.
    + name: nombre del fichero TarFile.
    + mode: modo de apertura del fichero. Soporta lecturas, escrituras y creación de archivos tanto para archivos como flujos de datos.
    + fileobj: para usar un objeto tipo fichero en vez de el nombre de un fichero. Debe ser un objeto tipo fichero binario.
    + bufsize: tamaño de bloques para la apertura, por defecto 20*512 bytes

ARCHIVACIÓN Y COMPRESIÓN DE FICHEROS - tarfile

Las funciones de tarfile (continuación):

- class tarfile.TarFile(name=None, mode='r', fileobj=None, format=DEFAULT_FORMAT, tarinfo=TarInfo, dereference= False, ignore_zeros=False, enconding=ENCODING, errors='surrogateescape', pax_headers=None, debug=0, errorlevel=0): objeto que representa el archivo TarFile para añadir, obtener o extraer contenido.
- TarFile.getmembers(): devuelve una lista de objetos tipo tarfile.TarInfo que representan el contenido de un archivo TarFile.
- TarFile.getmember(name): obtiene un archivo dado dentro del archivo TAR como instancia tarfile.TarInfo.
- TarFile.getnames(): devuelve una lista de nombres de archivos dentro del archivo TarFile.
- TarFile.extractall(path=".", members=None, numeric_owner=False): para extraer todos los miembros, o los especificados en members, del archivo TarFile. Se extrae el contenido dentro de la carpeta path.
- TarFile.extract(member, path="", set_attrs=True, numeric_owner=False): para extraer un único archivo member del archivo TarFile.
- TarFile.extractfile(member): devuelve un objeto tipo fichero de un miembro del archivo TarFile.
- TarFile.add(name, arcname=None, recursive=True, *, filter=None): para añadir el fichero especificado por name (fichero, directorio, ...) dentro del archivo TarFile. Con arcname se puede añadir con un nombre diferente.
- TarFile.addfile(tarinfo, fileobj=None): para añadir un objeto TarInfo especificado por tarinfo dentro del Tarfile. Si se pone fileobj, objeto fichero binario, se lee el contenido de dicho objeto para ser añadido.
- TarFile.close(): cierra el archivo TarFile y añade el contenido pendiente de guardar.

ARCHIVACIÓN Y COMPRESIÓN DE FICHEROS - tarfile

Ejemplo: guardamos los ficheros de los planetas y comprobamos el tamaño de los ficheros TAR generados con y sin compresión.

```
import tarfile

if __name__ == '__main__':
    ficheros_planetas = ['Mercurio.xml', 'Venus.xml', 'Tierra.xml', 'Marte.xml', 'Jupiter.xml',
                         'Saturno.xml', 'Urano.xml', 'Neptuno.xml']

    for sufijo, flags in [('.tar', 'w'), ('.tar.gz', 'w:gz'), ('.tar.bz', 'w:bz2'), ('.tar.xz', 'w:xz')]:
        tar = tarfile.open("planetas" + sufijo, flags)
        for nombre_fichero in ficheros_planetas:
            tar.add(nombre_fichero)
        tar.close()
```

```
natxo@natxo-TITAN:/media/natxo/KINGSTON/UNED_PYTHON/8. EyS_Ficheros$ ll *.tar*
-rw-r--r-- 1 natxo natxo 20480 jul 25 14:56 planetas.tar
-rw-r--r-- 1 natxo natxo   675 jul 25 14:56 planetas.tar.bz
-rw-r--r-- 1 natxo natxo   712 jul 25 14:56 planetas.tar.gz
-rw-r--r-- 1 natxo natxo   680 jul 25 14:56 planetas.tar.xz
```

La mejor compresión

Realiza la actividad ejemplo_tarfile.py